

UID EXPERIMENT-9

Name:Shanmuganathan S

Reg no.: 230701307

Step 1: Create the HTML Form

1. Email Field:

- Use a **<label>** for the email input.
- Use an **<input>** of type **email** with an ID and name.
- Add a **** for displaying error messages related to the email.

2. Phone Number Field:

- Use a **<label>** for the phone number input.
- Use an **<input>** of type **text** with an ID and name.
- Add a **** for displaying error messages related to the phone number.

3. Add a submit button to the form.

HTML Code:

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
  <title>Form Validation</title>
```

```
  <link rel="stylesheet" href="style.css">
```

```
</head>
```

```
<body>
```

```
  <div class="container">
```

```
    <form id="myForm">
```

```
      <label for="email">Email:</label>
```

```
<input type="email" id="email" name="email" required>
<span id="emailError" class="error"></span>

<label for="phone">Phone Number:</label>
<input type="text" id="phone" name="phone" required>
<span id="phoneError" class="error"></span>

<button type="submit">Submit</button>
</form>
</div>

<script src="https://cdnjs.cloudflare.com/ajax/libs/validator/13.6.0/validator.min.js"></script>
<script src="script.js"></script>
</body>
</html>
```

Step 2: Style the Form with CSS

1. Style Form Elements

- Use flexbox for the form layout.
- Add margins and padding to labels and input fields.
- Style the submit button with colors and hover effects.
- Style error messages to be red and appropriately sized.

CSS Code:

```
body {
  font-family: Arial, sans-serif;
  background-color: #f4f4f4;
  display: flex;
  justify-content: center;
  align-items: center;
  height: 100vh;
  margin: 0;
}
```

```
.container {  
  background-color: white;  
  padding: 20px;  
  border-radius: 5px;  
  box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);  
}
```

```
form {  
  display: flex;  
  flex-direction: column;  
}
```

```
label {  
  margin-bottom: 5px;  
}
```

```
input {  
  margin-bottom: 10px;  
  padding: 10px;  
  border: 1px solid #ccc;  
  border-radius: 3px;  
}
```

```
button {  
  padding: 10px;  
  background-color: #28a745;  
  color: white;  
  border: none;  
  border-radius: 3px;  
  cursor: pointer;
```

```
}
```

```
button:hover {  
    background-color: #218838;  
}
```

```
.error {  
    color: red;  
    font-size: 0.875em;  
}
```

Step 3: Add JavaScript for Validation

1. Add Event Listener for Form Submission

- Use **addEventListener** to listen for the form's submit event.

2. Prevent Default Submission

- Use **e.preventDefault()** to stop the form from submitting automatically.

3. Retrieve Input Values

- Get the values of the email and phone number input fields.

4. Clear Previous Error Messages

- Set the text content of the error message spans to an empty string.

5. Validate Email and Phone Number

- Use **validator.isEmail()** to check if the email is valid.
- Use **validator.isMobilePhone()** to check if the phone number is valid.
- If validation fails, display appropriate error messages.

6. Log Valid Inputs

- If both inputs are valid, log the email and phone number to the console.

Example JavaScript Code:

```
document.getElementById('myForm').addEventListener('submit', function (e) {  
    e.preventDefault();
```

```
    let email = document.getElementById('email').value;
```

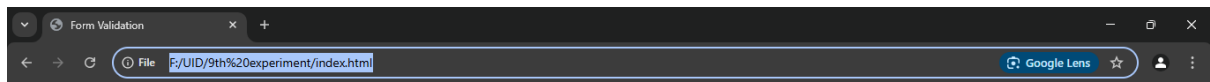
```
let phone = document.getElementById('phone').value;
let emailError = document.getElementById('emailError');
let phoneError = document.getElementById('phoneError');

// Clear previous error messages
emailError.textContent = "";
phoneError.textContent = "";

// Validate email
if (!validator.isEmail(email)) {
    emailError.textContent = 'Please enter a valid email address.';
}

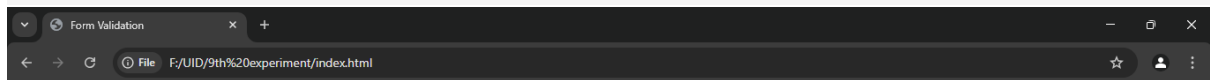
// Validate phone number
if (!validator.isMobilePhone(phone, 'any')) {
    phoneError.textContent = 'Please enter a valid phone number.';
}

// If no errors, submit the form (for demonstration purposes, we'll just log the values)
if (validator.isEmail(email) && validator.isMobilePhone(phone, 'any')) {
    console.log('Email:', email);
    console.log('Phone:', phone);
}
});
```



Email:

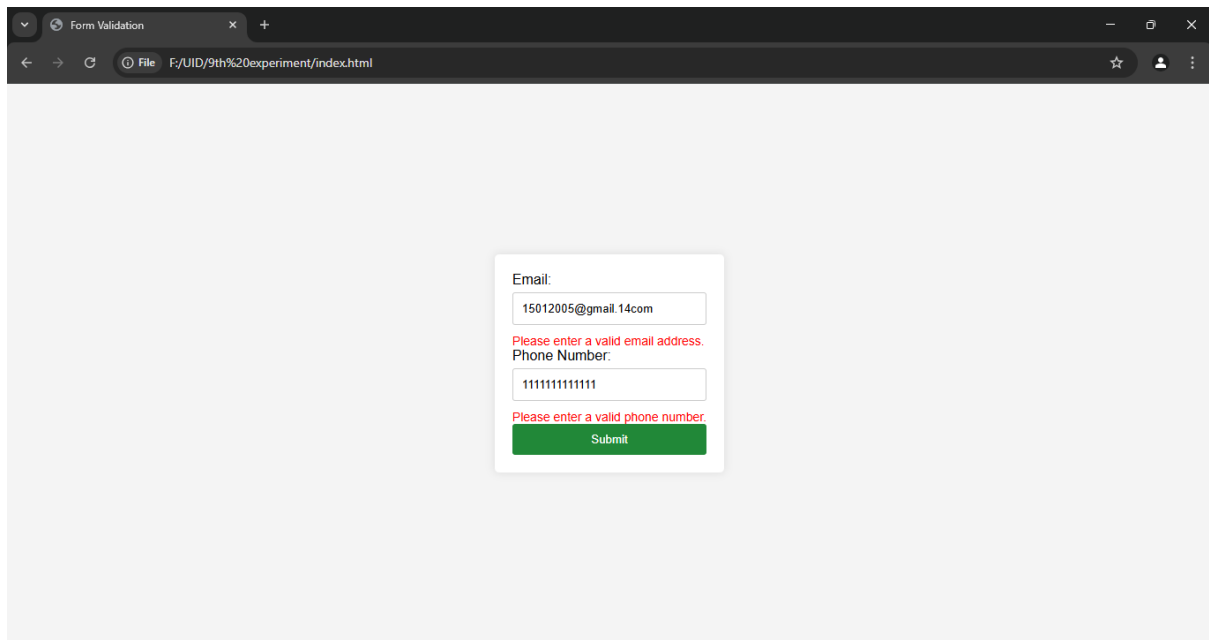
Phone Number:



Email:

Please enter a valid email address.

Phone Number:



Conclusion

By following these steps, you can successfully create a validated input form that checks for valid email and phone number entries, providing user feedback through error messages. This form can be further enhanced with additional features as needed.